

VAJRA

Visual Audio Joint-modal Rendering Architecture

VAJRA 2.0

Document 2 of 5

Deployment and Operations Guide

Installing, running and maintaining the platform

WHO THIS IS FOR

Whoever installs and administers the host

WHAT IT COVERS

- One-command install, and what each step does
- Pre-staging weights so 257 GB is not downloaded twice
- Air-gapped installation, and its one real trap
- Running as a service, backups, and troubleshooting

THE FULL SET

- | | |
|--|---------------------------|
| 1. System Requirements and Sizing | specifying the machine |
| ■ 2. Deployment and Operations Guide | installing and running it |
| 3. Operator's User Manual | using it |
| 4. Technical Writeup | understanding it |
| 5. Verification and Test Documentation | maintaining it |

Lt Col Raunak Malik · M.Tech, MCTE Mhow · IIT Indore

Supervisors: Dr. Nagendra Kumar (IIT Indore) · Dr. Krishan Berwal (MCTE Mhow)

Offline synthetic-media platform · for authorised research and capability development

1. What You Received

Item	Contents
Source archive	The complete application: 7 modules, setup scripts, tests, documentation. Small — under a megabyte.
MANIFEST.txt	Inside the archive: a SHA-256 for every file
models/ (separately)	Pre-fetched weights, if supplied on media
This document set	Five manuals, one per reader

Verify the transfer before installing

A truncated archive that still opens is a real failure mode when moving files between institutions, so the contents are checked rather than assumed:

```
unzip VAJRA_handover_2.0.zip
cd VAJRA
sha256sum -c MANIFEST.txt
```

Note. A SHA-256 detects **corruption**, not tampering: anyone who alters a file and recomputes the manifest produces a consistent archive. Detecting a forger needs a signature, which is a different threat model from the one a handover faces.

2. Install — the Short Version

Three commands on a prepared host. Expect the weight download to dominate the elapsed time.

```
python3 setup/preflight.py      # verify the machine first
bash install.sh voice lipsync   # core modules
bash run.sh                     # start it
```

Then open **http://localhost:7860** in a browser on that machine.

Choosing what to install

Command	Builds	Tabs it enables
bash install.sh	voice, lipsync (the default)	01 Video Edit, 02 Voice, 03 Image Gen, 04 Face Swap, 07 Media Studio
bash install.sh voice lipsync viitor	adds the local-infill engine	as above, plus tier-A quality in Video Edit
bash install.sh voice lipsync qwen	adds the image editor	adds 06 Image Edit
bash install.sh all	everything, including both video engines	all seven — but see the disk warning

Caution. **all** pulls the two video-generation engines, which together exceed 250 GB of weights. The installer prints this warning before starting. Install them only on a host sized for them.

3. What install.sh Actually Does

Five steps, in this order, and the order is the point — nothing large is downloaded until the machine has been shown capable of using it.

#	Step	What it does
1	preflight	Checks GPU, driver, both interpreters, tooling, disk and network. Fails here rather than an hour into a download.
2	venv_main	Creates the principal runtime in its own virtual environment. Nothing is installed into system Python.

#	Step	What it does
3	install_main.sh	ffmpeg and git-lfs via apt (the only privileged step), clones the third-party engines, installs the main requirements.
4	make_venvs.sh	Builds the isolated per-module environments, concurrently, each logging to its own file.
5	download_models.py	Fetches weights for the selected modules, skipping anything already present.

Why so many environments

The model families require mutually incompatible versions of the tensor, transformer and diffusion libraries — not merely inconvenient, actually incompatible. Each family therefore gets its own environment and is driven as a subprocess worker. The cost is disk; the benefit is that adding or upgrading one model cannot break another, which is what let the platform grow to seven modules.

It is also why **environments cannot be copied between machines**: they hardcode absolute paths and are built against a specific interpreter and CUDA runtime. A copied environment looks installed and fails at import, which is worse than being absent. They are rebuilt in minutes.

4. Pre-Staging the Weights

Weights, unlike environments, **are** portable — they are ordinary files. Copying them across saves the largest single cost of the install.

1. Copy your existing `models/` directory to the host, **including the `hf_cache/` sub-directory** — section 5 explains why that one matters.
2. Point the installer at it: `export VAJRA_MODELS=/data/vajra/models`
3. Run the install as normal.

The download step then becomes a verification pass. It checks before every fetch and reports `exists:` for anything already in place, so nothing is downloaded twice.

5. Air-Gapped Installation

If the host will never have internet, read this section before the transfer, not on the day.

The trap: four tabs fetch their weights on first use

Caution. The install-time downloader covers **only** the voice, lip-sync, face-swap and local-infill weights. The four largest models — image generation, image editing and both video engines — are fetched by their libraries the **first time that tab is used**, not during install. A `models/` directory copied from a machine that never ran those tabs will be missing them, and the tab will fail on an air-gapped host with a download error.

The fix is straightforward once known. All lazily-fetched weights land in one place, because the configuration redirects the Hugging Face cache into the model root:

```
HF_HUB_CACHE = $VAJRA_MODELS/hf_cache
```

Procedure

1. On a machine with internet — ideally the one you have been working on — run **every tab you intend to hand over at least once**. That populates `hf_cache/` with the four lazily-fetched model families.
2. Check what you actually have before packing: `du -sh $VAJRA_MODELS/*`. If `hf_cache` is a few gigabytes rather than tens, the large models are not there.
3. Copy the **whole** models directory, `hf_cache` included, to the transfer medium.
4. Pre-download the Python packages on a machine matching the target's OS, Python version and architecture — see the caution below.
5. On the host: copy everything into place, set `VAJRA_MODELS`, and install with `pip` pointed at the local wheel directory.

Caution. Python wheels are the hard part. They must be downloaded on a machine matching the target's operating system, Python version and CPU architecture. A Windows workstation cannot produce Linux CUDA wheels. If no matching machine is available, the practical answer is to obtain one-time network access on the host — it removes an entire class of problems.

6. Running It

```
bash run.sh          # http://localhost:7860
VAJRA_PORT=8080 bash run.sh  # a different port
```

Public sharing is **off** by default, deliberately: a public tunnel suits a disposable notebook session and is rarely what an institute wants on a machine that stays up. It can be enabled with `VAJRA_SHARE=1`, which needs internet and gives anyone holding the link full use of the system with no password.

As a service

To have it survive a reboot, create `/etc/systemd/system/vajra.service`:

```
[Unit]
Description=VAJRA studio
After=network.target

[Service]
Type=simple
User=vajra
WorkingDirectory=/opt/vajra
Environment=VAJRA_MODELS=/data/vajra/models
ExecStart=/bin/bash /opt/vajra/run.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now vajra
journalctl -u vajra -f          # watch the log
```

7. Routine Operation

Task	How
Where results are saved	outputs/, named vajra_<what>_<timestamp>
Reclaim disk	Clear old outputs/ and uploads/. Neither is self-clearing.
Back up	The archive plus models/. Environments are rebuilt, never backed up.
Add a module later	Re-run <code>bash install.sh</code> with the extra target. It is resumable and skips what exists.
Move to a new machine	Copy the archive and models/; rebuild environments there.
Confirm it is offline	Disconnect the network and run an edit end to end. This is the acceptance test worth doing in front of the recipient.

8. Troubleshooting

Symptom	Cause	Fix
preflight fails on python3.10	Only 3.12 present. The most common install failure.	<code>apt install python3.10 python3.10-venv python3.10-dev</code>
A module reports its environment is missing	That target was not built	Re-run <code>install.sh</code> with that target
Import errors inside one module only	A copied or half-built environment	Delete that directory under <code>venvs/</code> and re-run <code>install</code>
Download error on a tab, on an air-gapped host	A lazily-fetched model is not in <code>hf_cache</code>	See section 5 — the cache must be pre-populated
Out of memory on a video tab	System RAM, not VRAM — CPU offload needs the model to fit in host memory	Add RAM, or use a lower resolution and shorter duration
Second job appears to hang	GPU work is serialised by design	Wait. Nothing is wrong.
Interface unreachable	Not running, or firewalled	<code>systemctl status vajra</code> ; check the port

9. Responsible Operation

This platform produces synthetic media that can be mistaken for genuine recording. It is intended for authorised research and capability development in a controlled environment. It is offline by design so that material and results stay on the host.

Keep a record of what was generated and why; do not represent output as a genuine recording of any person; follow the institution's own authorisation and record-keeping rules. Access control is an institutional matter — the software does not authenticate its operator.